

# AI Smart Food Supply Orchestrator (ASF-Orchestrator) 상세 설계 문서 (LLD)

수요·공급·유통 동적 최적화를 위한 AI 기반 통합 오케스트레이터 플랫폼 고도화 설계

문서 번호 : ASF-ARCH-LLD-001

버전 : 1.0.0 (공식 배포본)

작성일자 : 2026년 6월 5일

작성자 : Generative AI Tech Lead / System Architect

배포 대상 : Core DevOps Team, AI Modeling Group, Logistics Operator

# 목차 (Table of Contents)

---

## 1. 모듈 아키텍처 및 세부 컴포넌트 설계

- 1.1 B2C API 백엔드 서버 (Go) 레이어드 아키텍처
- 1.2 AI 추론 및 파이프라인 (Python) 아키텍처
- 1.3 C++ VRP 솔버 래퍼 설계

## 2. 세부 트랜잭션 시퀀스 및 데이터 흐름

- 2.1 초개인화 AI 장바구니 추천 흐름 (B2C)
- 2.2 실시간 수급 연동 Dynamic Pricing 및 결제 프로세스
- 2.3 대규모 VRP 물류 경로 최적화 및 배치 작업 흐름 (B2B)

## 3. 물리 데이터 모델 및 캐싱 아키텍처 상세

- 3.1 PostgreSQL DDL 및 정규화/인덱스 상세 설계
- 3.2 Redis High-Performance 인메모리 스키마 구조

## 4. API 및 프로토콜 상세 스펙 (Contracts)

- 4.1 [B2C REST API] 장바구니 추천 조회
- 4.2 [B2B gRPC Protobuf] 최적 물류 경로 계산 서비스

## 5. 핵심 알고리즘 소스코드 레벨 구현

- 5.1 다목적 최적화 탐색 엔진 (Multi-Objective Greedy Optimizer)
- 5.2 Go API 아키텍처 핵심 데이터 구조체 및 데이터 흐름 제어 프레임워크

## 6. 프로젝트 디렉토리 레이아웃 및 빌드 구성

# 1. 모듈 아키텍처 및 세부 컴포넌트 설계

---

## 1.1 B2C API 백엔드 서버 (Go) 레이어드 아키텍처

---

Go 백엔드 서버는 높은 동시성과 저지연 데이터 처리를 위해 레이어드 아키텍처(Layered Architecture) 패턴을 따르며, 의존성 주입(Dependency Injection)을 활용하여 모듈 간 결합도를 최소화합니다.

- **Presentation Layer (Delivery):** HTTP 및 gRPC 엔드포인트를 노출하고, 클라이언트 요청 유효성 검증(Validation) 및 JSON/Protobuf 직렬화를 수행합니다.
- **Business Logic Layer (UseCase):** 핵심 비즈니스 규칙을 제어하며, 여러 리포지토리를 조합하여 하나의 트랜잭션 단위 작업을 조율합니다.
- **Data Access Layer (Repository):** PostgreSQL DB, Redis Cache, 외부 Triton 클라이언트 인터페이스 등 하위 인프라 스토리지로의 접근을 추상화합니다.

## 1.2 AI 추론 및 파이프라인 (Python) 아키텍처

---

AI 연산 엔진은 정기 시계열 수급 예측(TFT) 및 실시간 그래프 임베딩 추천(GraphSAGE) 처리를 분리하여 최적의 서버 인프라에 분산 매핑합니다.

- **Feature Store Client Interface:** Feast SDK를 래핑하여, DB나 GCS 파일 시스템에 접근하지 않고 Redis Online Store로부터 예측 피쳐 벡터를 실시간 패치합니다.
- **Triton Client Worker:** Go 서버 또는 Python 워커로부터 추론 요청이 인입될 때 Triton Inference Server의 gRPC 인터페이스를 가동하여 모델을 병렬 호출하고, 예측 텐서를 수신합니다.

## 1.3 C++ VRP 솔버 래퍼 설계

---

대용량 공간 경유지 최적화 연산은 Go/Python 환경보다 네이티브 메모리 및 SIMD 명령어가 제공되는 C++ 코어 모듈을 가동합니다.

- **Data Model Mapping:** 위경도 데이터를 받아 C++ 구조체 **Node** 및 **Vehicle** 인스턴스로 파싱해 맵핑합니다.
- **Solver DLL/SO Integration:** Go 언어의 Cgo 바인딩 기술을 적용하거나 독립 마이크로서비스 gRPC 데몬 형태로 컴파일하여 런타임 결합도를 낮추고 격리 배포합니다.

## 2.1 초개인화 AI 장바구니 추천 흐름 (B2C)

```
[User Client]      [Go API Server]      [Redis Cache]      [Triton AI Server]      [Feast / DB]
```

```
|                  |                  |                  |                  |
```

```
| -- GET /basket --->|                  |                  |                  |
```

```
|                  |--- 1. Check 캐시 ->|                  |                  |
```

```
|                  | <-- 캐시 미스 -----|                  |                  |
```

```
|                  |                  |                  |                  |
```

```
|                  |--- 2. 피쳐 데이터 Fetch ----->|                  |                  |
```

```
|                  | <-- 피쳐 벡터 수신 -----|                  |                  |
```

```
|                  |                  |                  |                  |
```

```
|                  |--- 3. 텐서 변환 및 추론 gRPC 호출 ---->|                  |                  |
```

```
|                  | <-- 추론 예측 결과 수신 -----|                  |                  |
```

```
|                  |                  |                  |                  |
```

```
|                  |--- 4. 다목적 최적화 스코어링 연산 -----|                  |                  |
```

```
|                  |                  |                  |                  |
```

```
|                  |--- 5. 결과 캐시 쓰기 ---->|                  |                  |
```

```
|                  |                  |                  |                  |
```

```
| <- 200 OK (JSON) --|                  |                  |                  |
```

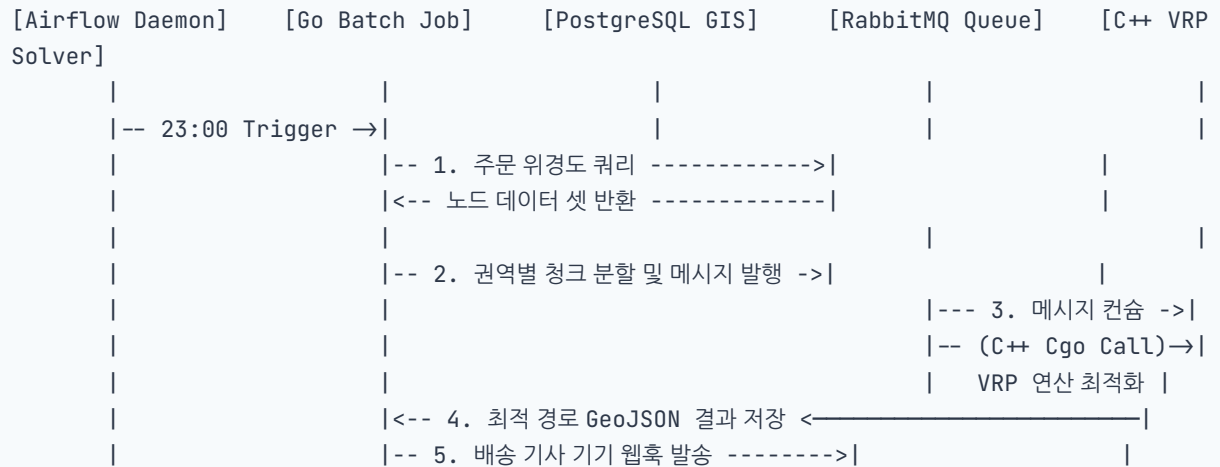
```

[User Client]      [Go API Server]      [Redis Lock]      [PostgreSQL DB]      [Message
Broker]
|                  |                  |                  |                  |
|-- POST /checkout ->|                  |                  |                  |
|                  |--- 1. 분산락 획득 ->|                  |                  |
|                  |← Lock OK -----|                  |                  |
|                  |                  |                  |                  |
|                  |--- 2. DB 트랜잭션 개시 ----->|                  |
|                  |   - 유저 지갑 차감                  |                  |
|                  |   - 수급 기여 리워드 포인트 적립      |                  |
|                  |   - 주문 기록 삽입                  |                  |
|                  |←-- 트랜잭션 커밋 완료 -----|                  |
|                  |                  |                  |                  |
|                  |--- 3. 분산락 해제 ->|                  |                  |
|                  |                  |                  |                  |
|                  |--- 4. 비동기 공급망 재고 차감 이벤트 전송 ----->|

```

## 2.3 대규모 VRP 물류 경로 최적화 및 배치 작업 흐름 (B2B)

주문 수집이 완료된 23:00 정각에 기동하여 유통망 배차 경로를 연산해 공급망에 전파하는 지연 없는 오케스트레이션 설계입니다.



## 3. 물리 데이터 모델 및 캐싱 아키텍처 상세

### 3.1 PostgreSQL DDL 및 정규화/인덱스 상세 설계

인증, 상품 마스터, 예측 수급 가격, 그리고 GIS 처리를 위한 고성능 공간 데이터 모델 설계 명세입니다.

```
-- 1. 사용자 마스터데이터 및 보안 인증 정보 테이블
CREATE TABLE users (
    user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    household_size INT NOT NULL DEFAULT 1 CHECK (household_size > 0),
    health_target_keywords VARCHAR(50)[] DEFAULT '{}',
    monthly_budget DECIMAL(10, 2) DEFAULT 500000.00,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_users_email ON users(email);

-- 2. 농산물 정보 마스터 테이블
CREATE TABLE agricultural_products (
    product_code VARCHAR(20) PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    category_name VARCHAR(50) NOT NULL,
    origin_location VARCHAR(100) NOT NULL,
    base_unit VARCHAR(10) NOT NULL,
    base_price DECIMAL(10, 2) NOT NULL CHECK (base_price ≥ 0),
    oversupply_risk_index DECIMAL(3, 2) NOT NULL DEFAULT 0.00 CHECK
(oversupply_risk_index BETWEEN 0.00 AND 1.00),
    carbon_emissions_factor DECIMAL(5, 2) NOT NULL DEFAULT 0.00,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_products_category ON agricultural_products(category_name, is_active);
CREATE INDEX idx_products_oversupply ON agricultural_products(oversupply_risk_index
DESC) WHERE is_active = TRUE;

-- 3. 고성능 공간 지리 데이터 연동 주문서 테이블
CREATE TABLE orders (
    order_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE RESTRICT,
    total_amount DECIMAL(12, 2) NOT NULL,
    discount_amount DECIMAL(12, 2) NOT NULL DEFAULT 0.00,
    earned_esg_points INT NOT NULL DEFAULT 0,
    delivery_address VARCHAR(255) NOT NULL,
    delivery_coordinate GEOMETRY(Point, 4326) NOT NULL,
```

```
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 공간 쿼리 속도 보장을 위한 GiST 인덱스
CREATE INDEX idx_orders_delivery_coordinate ON orders USING GIST(delivery_coordinate);
CREATE INDEX idx_orders_user_created ON orders(user_id, created_at DESC);
```

## 3.2 Redis High-Performance 인메모리 스키마 구조

밀리초(ms) 단위의 락킹 및 읽기 트래픽 처리를 위해 Redis 내부 필드 형식을 바이트 스트림 레벨로 엄격히 관리합니다.

- 동적 가격 및 할인 정보 해시 스키마:
  - 키 포맷: `product:pricing:{product_code}` (TTL: 1800초)
  - 데이터 구조: Hash ( `p_price` : float, `d_rate` : float, `risk` : float, `up_ts` : integer)
- 분산락 구현을 위한 Key-Value 스펙:
  - 키 포맷: `lock:order:checkout:{user_id}` (TTL: 5초)
  - 데이터 구조: String (Value: 고유 UUID 문자열, NX PX 옵션을 적용하여 상호 배제 보장)

## 4. API 및 프로토콜 상세 스펙 (Contracts)

### 4.1 [B2C REST API] 장바구니 추천 조회

GET /api/v1/recommendation/basket

#### Request Headers

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...  
Accept: application/json

#### Response Body (HTTP 200 OK)

```
{
  "status": "success",
  "data": {
    "basket_id": "c73a884d-2a1f-4cf6-ba64-50a7c936df31",
    "calculated_at": "2026-06-05T15:20:00Z",
    "summary": {
      "total_items_count": 2,
      "estimated_original_price": 7500.00,
      "estimated_discounted_price": 5550.00,
      "total_discount_rate": 0.26,
      "estimated_esg_points": 350
    },
    "items": [
      {
        "product_code": "AGRI-CABB-001",
        "product_name": "유기농 괴산 양배추",
        "quantity": 1,
        "original_price": 3000.00,
        "discounted_price": 2400.00,
        "oversupply_risk_level": "HIGH",
        "esg_score_contribution": 150
      },
      {
        "product_code": "AGRI-ONIO-053",
        "product_name": "햇 양파",
        "quantity": 1,
        "original_price": 4500.00,
        "discounted_price": 3150.00,
        "oversupply_risk_level": "VERY_HIGH",
        "esg_score_contribution": 200
      }
    ]
  }
}
```



```
}  
}
```

## 4.2 [B2B gRPC Protobuf] 최적 물류 경로 계산 서비스

```
syntax = "proto3";  
  
package logistics.v1;  
  
service LogisticsService {  
    rpc CalculateRoute(CalculateRouteRequest) returns (CalculateRouteResponse);  
}  
  
message Coordinate {  
    double latitude = 1;  
    double longitude = 2;  
}  
  
message DeliveryDestination {  
    string destination_id = 1;  
    Coordinate coordinate = 2;  
    double demand_weight_kg = 3;  
}  
  
message CalculateRouteRequest {  
    string warehouse_id = 1;  
    Coordinate warehouse_coordinate = 2;  
    repeated DeliveryDestination destinations = 3;  
    double vehicle_max_capacity_kg = 4;  
}  
  
message OptimizedRoute {  
    int32 route_index = 1;  
    repeated string path_destination_ids = 2;  
    double total_distance_meters = 3;  
    double total_travel_time_seconds = 4;  
    double co2_emitted_kg = 5;  
}  
  
message CalculateRouteResponse {  
    string warehouse_id = 1;  
    repeated OptimizedRoute optimized_routes = 2;  
    double base_co2_reduction_percentage = 3;  
}
```

## 5. 핵심 알고리즘 소스코드 레벨 구현

### 5.1 다목적 최적화 탐색 엔진 (Multi-Objective Greedy Optimizer)

아래 Python 스크립트는 소비자 선호, 공급 과잉 및 푸드마일리지를 모두 고려한 다목적 장바구니 선정 모델의 세부 구현체입니다.

```
import numpy as np

def compute_user_nutrition_utility(user_id: str, product_code: str) → float:
    # 모크용 개인화 유틸리티 산출 점수 (실제 구현에서는 임베딩 내적 혹은 룰 필터링 연동)
    hash_val = hash(user_id + product_code) % 100
    return float(hash_val) / 100.0

def calculate_geographical_distance_km(origin_loc: str) → float:
    # 물류센터 혹은 배송지까지의 간이 기하적 거리 반환
    # 실제 운영 환경: PostGIS 하버사인 공간 계산 연동
    return 15.0 if "괴산" in origin_loc else 120.0

def select_optimized_basket(user_id: str, candidate_items: list, k_limit: int,
                           alpha: float, beta: float, gamma: float) → list:
    scored_items = []

    for item in candidate_items:
        # 1. 개인화 영양 궁합 스코어링
        utility = compute_user_nutrition_utility(user_id, item['product_code'])

        # 2. 산지 및 물류 마일리지 패널티 계산
        distance = calculate_geographical_distance_km(item['origin_location'])
        # 거리가 길수록 이산화탄소 배출이 커지므로 정규화된 패널티 스코어로 적용
        food_mile_penalty = min(distance / 500.0, 1.0)

        # 3. 공급 과잉 지수 매핑
        oversupply = item['oversupply_risk_index']

        # 4. 목적함수 수식 계산:  $f(u, i) = a \cdot Util + b \cdot Over - g \cdot Mile$ 
        total_score = (alpha * utility) + (beta * oversupply) - (gamma * food_mile_penalty)

        scored_items.append({
            "product_code": item['product_code'],
            "score": round(total_score, 4),
            "utility": round(utility, 2),
            "oversupply": round(oversupply, 2),
            "food_mile_penalty": round(food_mile_penalty, 2)
        })

    # 점수 역순으로 정렬
```

```

    scored_items.sort(key=lambda x: x['score'], reverse=True)
    return scored_items[:k_limit]

# 테스트 러너
if __name__ == "__main__":
    test_candidates = [
        {"product_code": "AGRI-CABB-001", "origin_location": "충북 괴산",
"oversupply_risk_index": 0.85},
        {"product_code": "AGRI-ONIO-053", "origin_location": "전남 무안",
"oversupply_risk_index": 0.95},
        {"product_code": "AGRI-PORK-102", "origin_location": "경기 이천",
"oversupply_risk_index": 0.20},
    ]
    result = select_optimized_basket("user_dev_01", test_candidates, 2, alpha=0.4,
beta=0.5, gamma=0.1)
    print("선택된 추천 장바구니 리스트:", result)

```

## 5.2 Go API 아키텍처 핵심 데이터 구조체 및 데이터 흐름 제어 프레임 워크

Go 백엔드 도메인 계층 내 데이터 전송 객체(DTO) 정의 및 레포지토리 의존성 매핑 바인딩 소스코드입니다.

```

package domain

import (
    "context"
    "time"
)

// Product 대표 도메인 구조체
type Product struct {
    ProductCode      string    `json:"product_code" db:"product_code"`
    ProductName      string    `json:"product_name" db:"product_name"`
    CategoryName     string    `json:"category_name" db:"category_name"`
    OriginLocation   string    `json:"origin_location" db:"origin_location"`
    BaseUnit         string    `json:"base_unit" db:"base_unit"`
    BasePrice        float64   `json:"base_price" db:"base_price"`
    OversupplyRiskIndex float64   `json:"oversupply_risk_index" db:"oversupply_risk_index"`
    CarbonEmissions  float64   `json:"carbon_emissions_factor" db:"carbon_emissions_factor"`
    IsActive         bool      `json:"is_active" db:"is_active"`
    CreatedAt        time.Time `json:"created_at" db:"created_at"`
}

// User 대표 도메인 구조체
type User struct {
    UserID           string    `json:"user_id" db:"user_id"`

```

```

        Email            string    `json:"email" db:"email"`
        PasswordHash     string    `json:"- " db:"password_hash"`
        HouseholdSize     int       `json:"household_size" db:"household_size"`
        HealthTargetKeywords []string `json:"health_target_keywords"
db:"health_target_keywords"`
        MonthlyBudget     float64   `json:"monthly_budget" db:"monthly_budget"`
    }

// BasketUseCase 비즈니스 도메인 서비스 인터페이스
type BasketUseCase interface {
    GetOptimizedBasket(ctx context.Context, userID string) (*BasketResponse, error)
}

// ProductRepository 인프라스트럭처 레포지토리 인터페이스
type ProductRepository interface {
    FetchActiveCandidates(ctx context.Context) ([]Product, error)
    GetByCode(ctx context.Context, code string) (*Product, error)
}

type BasketItem struct {
    ProductCode      string    `json:"product_code"`
    ProductName      string    `json:"product_name"`
    Quantity         int       `json:"quantity"`
    OriginalPrice     float64   `json:"original_price"`
    DiscountedPrice   float64   `json:"discounted_price"`
    OversupplyRiskLevel string    `json:"oversupply_risk_level"`
    EsgScoreContribution int       `json:"esg_score_contribution"`
}

type BasketResponse struct {
    BasketID      string    `json:"basket_id"`
    CalculatedAt time.Time `json:"calculated_at"`
    Items         []BasketItem `json:"items"`
}

```

## 6. 프로젝트 디렉토리 레이아웃 및 빌드 구성

마이크로서비스로서의 이식성과 유지보수 편의성을 보장하기 위해 클린 아키텍처 및 Go/Python 표준 레이아웃을 준수합니다.

```
asf-orchestrator-root/
├── cmd/
│   └── api-server/           # Go REST/gRPC 메인 엔트리 포인트
│       └── main.go
├── internal/                 # Go 비즈니스 코어 패키지 (클린 아키텍처 구조)
│   ├── delivery/            # HTTP 핸들러 및 gRPC 서버 구현체
│   │   ├── http/
│   │   └── grpc/
│   ├── usecase/              # 비즈니스 오케스트레이션 레이어
│   ├── repository/           # DB / Redis / Triton 드라이버 클라이언트 연동
│   └── domain/                # 엔티티 및 공통 인터페이스 스펙
├── ai-engines/               # Python 인공지능 예측 및 추천 엔진 레포지토리
│   ├── models/               # TFT, GraphSAGE 가중치 및 직렬화 파일 (.onnx/.pt)
│   ├── config/               # Triton Model Config 파일 (config.pbtxt)
│   ├── train/                # 파이프라인 학습 소스코드
│   └── feature_store/         # Feast 피쳐 정의 스펙 (feature_services.yaml)
├── solver-vrp/               # C++ 기반 고속 배차 경로 최적화 코어 모듈
│   ├── src/
│   │   ├── main.cpp
│   │   └── vrp_optimizer.cpp
│   └── CMakeLists.txt
├── docker-compose.yml        # 로컬 다중 컨테이너 테스트 오케스트레이션 구성 파일
└── Makefile                  # 로컬 빌드, DB 마이그레이션 및 자동 린트 유틸리티
```